

Deploy Rozgar on Vercel

Two projects, one GitHub repository, the right environment variables, and a safe way to prove the website works.

Current production projects: **jobsi** serves the website and **jobs** serves the API. Both are already deployed. Follow this guide to understand and maintain that setup. The 52 source schedules are daily; prompt alerts and external monitoring still require the services explained below.

This document intentionally contains no credential values. Never copy secrets into a PDF, commit, browser screenshot, or chat.

1. What is deployed?

Rozgar has a Next.js frontend in `Frontend/` and a FastAPI backend in `Backend/`. They are folders in one GitHub repository and appear as **two separate Vercel projects**. A browser visits the website, which calls the API over HTTPS. The API reads and writes PostgreSQL.

Project	Root Directory	Current production URL	Role
<code>codeaza1/jobsi</code>	<code>Frontend</code>	https://jobsi-ten.vercel.app	Next.js website
<code>codeaza1/jobs</code>	<code>Backend</code>	https://jobs-codeaza1.vercel.app	FastAPI and Cron endpoints

The website forwards its `/api/jobs`, `/api/saved-searches`, and `/api/health` paths to the backend. Browser requests remain on the website origin, including temporary Vercel deployment URLs. Old deployment URLs keep their original code; use the current production alias or the newest Ready deployment when checking a fix.

Keep the two projects. In each Vercel project, open **Settings** → **Git** and confirm the same Rozgar repository and `main` production branch. Open **Settings** → **Build and Deployment** and confirm its Root Directory. A separate GitHub repository for each project is possible, but the current shared-repository structure is simpler and already working.

2. Terms and safety

An **environment variable** is a named setting that the running app reads. A **secret** is a value such as a database password or API key. `NEXT_PUBLIC_` variables are delivered to the browser, so never put a password in one. A **deployment** is a snapshot of code plus the environment selected at build time. Changing a Vercel variable requires a **new deployment**.

The repository has local `Backend/.env` and `Frontend/.env.local`. These are local-only and must remain ignored. Set production values in the matching Vercel project, from the service dashboards. Do not copy unverified development credentials into production.

3. Frontend variables: codeaza1/jobsi

Open Vercel `jobsi` → **Settings** → **Environment Variables**. Choose **Production** for the production site. Check or add each name below. Enter raw values without surrounding quotes; for URLs, avoid a trailing slash.

Name	What value to use / where to find it	Status on 15 Sep
<code>NEXT_PUBLIC_API_URL</code>	<code>https://jobs-codeaza1.vercel.app</code> ; the live backend URL.	Present and checked
<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code>	Publishable key from your chosen Clerk instance.	Present
<code>CLERK_SECRET_KEY</code>	Secret key from that same Clerk instance; mark Sensitive.	Present

NEXT_PUBLIC_EMAIL_ALERTS_ENABLED	false until a real alert is sent and received.	Added; requires new frontend deployment
NEXT_PUBLIC_SENTRY_DSN	Frontend DSN from your Sentry project, if you use Sentry.	Missing; optional until service setup
SENTRY_DSN	Server DSN from Sentry, if enabled.	Missing; optional until service setup

If Clerk shows a development badge, use one consistent Clerk environment across frontend and backend before public release. Do not mix a development frontend key with a production backend issuer.

4. Backend variables: codeaza1/jobs

Open **jobs** → **Settings** → **Environment Variables** → **Production**. Check this table. Keep secrets Sensitive. Values shown as “present” were checked by key name or non-secret URL match; no secret values were printed.

Name	What it means	Status
APP_ENV	production ; turns on the app's production safeguards.	Added; requires new backend deployment
DATABASE_URL	Neon pooled PostgreSQL connection string, including password; use your own Neon dashboard.	Present
FRONTEND_URL	https://jobsi-ten.vercel.app ; link in alerts.	Present and matched
CORS_ORIGINS	https://jobsi-ten.vercel.app ; allowed browser website.	Present and contains live site
CLERK_AUTHORIZED_PARTIES	Allowed frontend origin; include exact live URL.	Present and contains live site
CLERK_ISSUER , CLERK_SECRET_KEY	Same Clerk instance as the frontend; issuer URL and secret.	Present
CRON_SECRET	Long random bearer secret for Vercel Cron; keep private.	Present
SCRAPE_SECRET_KEY	Private key for administrative scrape endpoints.	Present
ADMIN_USER_IDS	Clerk user IDs permitted to inspect admin health, if needed.	Missing
RESEND_API_KEY , SENDER_EMAIL	Resend key and a verified sender address for email alerts.	Missing; no real email delivery
REDIS_URL	Remote serverless Redis URL reachable from Vercel; localhost cannot work.	Missing; cache falls back
SENTRY_DSN	Backend Sentry DSN for error reports.	Missing; external reports off

5. Save, redeploy, and check

1. Save each new variable on the correct project with **Production** selected. Keep Preview values separate if you use preview deployments.
2. Open each project's **Deployments** page and redeploy the newest main deployment, or push a new commit to main . Wait for **Ready** in both projects. Check the production alias, not only the temporary

deployment URL.

3. Visit the [website](#) on phone and desktop. Search for a word, open a job, and click the original application URL. Try light/dark mode and shortlisting.
4. Open [API live check](#) and [API ready check](#). The former shows the function responds; the latter also checks database readiness. Open [a jobs search](#) (or enter `/jobs?keyword=Python&summary=true` after the API URL).
5. Sign in, save a search, refresh, and check it remains yours. If sign-in fails, recheck that frontend and backend Clerk settings belong to the same instance.

The link to a jobs search above may be URL encoded by PDF readers. If it does not work, type `https://jobs-codeaza1.vercel.app/jobs?keyword=Python&summary=true` directly in your browser.

6. Scraping and alerts

The backend's `vercel.json` defines **53 protected daily Cron paths**: one for each of 52 public source sites and one alert check. Vercel sends `CRON_SECRET` as a bearer token. On a Hobby plan the schedule is at most daily and its exact hour can vary. Check **jobs** → **Cron Jobs** and deployment logs after a full cycle; a registered path alone does not prove all 52 sources ran successfully. Each employer board was live-validated when added, but ongoing reliability must be measured over days.

If your lead requires a new matching job to trigger an alert “the moment it appears,” run the separate Docker/APScheduler worker on an approved always-on Codeaza host. Daily Vercel Cron cannot promise that timing. Configure the worker from `Backend/.env.example`, keep it connected to the same database, and confirm the host is really running. The current host/access is not documented or verified.

Saved searches can be stored without email delivery. To enable email: verify a domain/sender in Resend, add `RESEND_API_KEY` and `SENDER_EMAIL` to backend Production, redeploy, then create a test saved search matched by a new job and confirm a real inbox receives exactly one email. Only after that check, set frontend `NEXT_PUBLIC_EMAIL_ALERTS_ENABLED=true` and redeploy. WhatsApp requires a separately approved provider and is not implemented.

7. Health, errors, and real users

Redis and Sentry code paths are optional until you provide real remote service URLs. Add `REDIS_URL` and Sentry DSNs, redeploy, then prove a cache hit and an error event appear in their dashboards. Add `ADMIN_USER_IDS` before using the authenticated health dashboard. For Week 8, recruit actual consenting users, choose a metric such as successful job-detail-to-apply clicks, record a baseline, make an improvement, then report the before/after result. Code and a demo script do not replace this evidence.

8. Troubleshooting

Symptom	First check
Page loads but has no jobs	Frontend <code>NEXT_PUBLIC_API_URL</code> , backend readiness, browser network response, and a recent scrape run.
Browser says CORS error	Backend <code>CORS_ORIGINS</code> exact frontend origin, then redeploy backend.

Sign-in or saved search fails	Both Clerk keys, issuer and authorized parties must refer to the same instance/origin.
Variable changed but nothing changed	Redeploy the affected project; confirm the production alias now points to the new Ready deployment.
Backend says database unavailable	Neon database state, pooled <code>DATABASE_URL</code> , Vercel function logs.
Cron returns 401	<code>CRON_SECRET</code> exists in backend Production; inspect Cron logs, never publish its value.
A source says never run	Check Cron Jobs registrations and wait for first daily cycle or run an authorized private scrape.
No alert email	Resend key/sender verified, backend deployed with them, matching saved search, alert run and delivery logs.
No cache or error event	External Redis/Sentry credentials and dashboards are still missing.

9. Rollback and official help

If a new deployment breaks the site, open **Deployments**, choose the previous Ready version, and use Vercel's rollback/alias controls. First check whether a database migration changed schema: application rollback does not automatically reverse database changes. Keep the private service keys unchanged unless the incident involves a compromised secret.

- [Vercel monorepos and root directories](#)
- [Vercel environment variables and redeploy behavior](#)
- [Vercel Cron jobs and secret authentication](#)
- [Vercel Cron plan limits and timing](#)
- [Ashby public job API](#) and [Greenhouse public job board API](#)

Repository companion files: `README.md`, `docs/deployment.md`, `docs/week-8-release.md`, `Backend/.env.example`, and `Frontend/.env.example`. The `project/testing/interview` PDF explains each feature and the remaining roadmap proof.